PROJECT REPORT OF INDUSTRY ORIENTED HANDS-ON EXPERIENCE (IOHE)
ON

AetherGuard-Forensi

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted By	Roll No.	Supervised By	Organization
Nishant	2210991988	Dr Rajat Takkar	Chitkara University
Anshul Mishra	2210991311		
Piyush Bhardwaj	2210992041		
Nitin Choudhary	2210991999		

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDI

ACKNOWLEDGEMENT

I hereby certify that the work which is being presented in the project report entitled **"AetherGuard-Forensi"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Rajat Takkar** The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **Nishant**

Date: 30-04-2026 2210991988

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr Rajat Takkar

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ABSTRACT

AetherGuard Forensic is an advanced AI-powered forensic platform designed to combat the rising threat of deepfake media and synthetic digital manipulation. The system provides unified multimodal analysis across images, videos, and live webcam streams, enabling investigators and organizations to verify content authenticity with high precision. Leveraging EfficientNet-B4 ensembles, Vision Transformers, and MTCNN-based facial analysis, AetherGuard detects generative artifacts, biometric anomalies, and facial inconsistencies in real time.

The platform integrates a scalable React + FastAPI architecture with PyTorch-driven inference pipelines, offering enterprise-grade features such as fraud probability scoring, persistent investigation history through the Vault Persistence Layer, and live forensic dashboards with latency and fraud ratio monitoring. Additional capabilities include forensic report generation, multimedia scalability, and cinematic glassmorphism-based UI for immersive investigative workflows.

By bridging the research gap in unified forensic ecosystems, AetherGuard delivers a complete solution for modern digital investigations, enhancing trust, transparency, and operational efficiency in the fight against misinformation and AI-generated threats.

Built using a scalable React + FastAPI architecture with PyTorch-powered AI models, AetherGuard delivers a complete digital forensic ecosystem optimized for deepfake detection, multimedia intelligence, and next-generation AI-powered cyber investigations.

TABLE OF CONTENTS

S.No.	Title	Page No.
1	Acknowledgement	2
2	Abstract	3
4	Table of contents	4
4	Introduction	5-8
5	Methodology	9-11
6	System Design and Architecture	12-14
7	Implementation	15-16
8	Major Findings/Outcomes/Output/Results	17-22
9	Conclusion and Future Scope	23
10	References	24
11	Appendices	25

1. Introduction

1.1. Background and Context

The rise of deepfake technology has made synthetic media almost indistinguishable from authentic content. While these tools have creative applications, they also pose serious risks such as misinformation, fraud, and identity theft. Existing forensic solutions are fragmented, focusing only on limited tasks like static image analysis, and lack real-time monitoring, biometric validation, and persistent investigation tracking.

While deepfakes have potential applications in entertainment, education, and creative industries, they also pose serious risks. Malicious actors exploit them to spread misinformation, commit fraud, impersonate individuals, and undermine trust in digital communication. The increasing accessibility of deepfake generation tools has amplified these threats, creating challenges for investigators, enterprises, and governments worldwide.

Existing forensic solutions often operate in silos, focusing on narrow tasks such as static image analysis or basic classification. They lack real-time monitoring, biometric validation, forensic reporting, and persistent investigation tracking, forcing organizations to rely on multiple disconnected tools. This fragmented approach results in inefficiency, inconsistent analysis, and limited transparency.

AetherGuard-Forensi addresses this gap by offering a unified AI-powered forensic ecosystem. It combines multimodal analysis of images, videos, and live webcam streams with transformer-based intelligence and biometric anomaly detection. With enterprise dashboards, fraud probability analytics, and investigation history management, the platform empowers investigators and organizations to restore trust in digital communication and effectively combat synthetic media threats.

1.2. Problem Statement

The rapid growth of deepfake technology has made synthetic media highly realistic and difficult to distinguish from authentic content. This has led to increased risks of misinformation, fraud, and identity manipulation across digital platforms. Current forensic tools are fragmented, focusing only on limited tasks such as static image analysis, and lack real-time monitoring, and persistent investigation tracking.

There is a clear need for a **unified forensic platform** that can provide multimodal deepfake detection across images, videos, and live streams, while offering enterprise-grade analytics and secure investigation history management.

1.3. Objectives

- Develop a unified AI-powered forensic platform for detecting deepfakes in images, videos, and live webcam streams.
- Provide real-time monitoring, fraud probability scoring, and biometric anomaly detection.
- Enable persistent investigation tracking through secure forensic storage (Vault Persistence Layer).
- Deliver enterprise-grade dashboards with latency metrics, fraud ratios, and forensic activity logs.
- Reduce manual effort and improve efficiency in digital investigations through automation and scalability.

1.4. Proposed Solution / Overview

AetherGuard-Forensi is designed as a unified AI-powered forensic platform that integrates multimodal deepfake detection across images, videos, and live webcam streams. Unlike existing fragmented tools, it provides investigators and organizations with a single ecosystem for digital media verification.

The solution leverages EfficientNet-B4 ensembles, Vision Transformers, and MTCNN-based facial analysis to identify generative artifacts, biometric anomalies, and facial inconsistencies with high accuracy. A scalable React.js frontend and FastAPI + PyTorch backend ensure real-time performance, while the Vault Persistence Layer maintains investigation history and forensic logs for long-term evidence tracking.

Key capabilities include:

- Real-time forensic monitoring and fraud probability analytics
- Persistent investigation storage and report generation
- Enterprise dashboards with latency metrics and fraud ratios
- Scalable multimedia processing and responsive UI with cinematic glassmorphism design

By combining advanced AI inference with enterprise grade monitoring, AetherGuard Forensi delivers a complete forensic ecosystem that enhances trust, transparency, and efficiency in combating synthetic media threats.

1.5. Expected Outcome

- **Accurate Deepfake Detection**: Reliable identification of manipulated content across images, videos, and live webcam streams.
- **Enhanced Media Verification**: Improved authenticity checks through AI-powered biometric anomaly and artifact detection.
- **Real-Time Monitoring**: Continuous forensic activity tracking with fraud probability analytics and latency metrics.
- **Centralized Investigation Management**: Persistent storage of forensic logs and investigation history via the Vault Persistence Layer.
- **Operational Efficiency**: Reduced manual effort and faster investigation workflows through automation and scalable multimedia processing.
- **Enterprise-Grade Insights**: Interactive dashboards providing fraud ratios, forensic statistics, and live analytics for investigators and organizations

1.6. Original Contribution

- **Unified Ecosystem**: Unlike existing fragmented tools, *AetherGuard-Forensi* integrates image, video, and live webcam deepfake detection into a single platform.
- **Advanced AI Models**: Introduces ensemble analysis using EfficientNet-B4 and Vision Transformers, combined with MTCNN-based biometric validation, for higher accuracy in detecting generative artifacts and facial inconsistencies.
- **Persistent Investigation Tracking**: Implements the Vault Persistence Layer to maintain forensic logs and investigation history, ensuring transparency and long-term evidence management.
- **Enterprise-Grade Features**: Provides real-time fraud probability scoring, latency monitoring, and interactive dashboards for investigators and organizations.
- **Immersive UI Design**: Incorporates cinematic glassmorphism-based architecture for a modern, responsive, and user-friendly forensic interface

Feature Area	Existing Tools/Platforms	Traditional Detection Systems	Basic AI Detectors	AetherGuard Forensic (Proposed)
Image Deepfake Detection	Basic	Limited	Good	**Excellent**
Video Deepfake Analysis	Limited	None	Basic	**Excellent**
Live Webcam Detection	None	None	None	**Real-Time Detection**
Biometric Anomaly Detection	None	Basic	Limited	**AI-Powered Analysis**
Transformer-Based AI Analysis	None	None	Limited	**Integrated Ensemble**
Investigation History Tracking	None	None	None	**Vault Persistence**
Unified Multimedia Verification	None	None	Partial	**Yes**
Modern Enterprise UI	Basic	Traditional	Basic	**Cinematic Glassmorphism**

2. Methodology

The methodology adopted for *AetherGuard-Forensi* focuses on designing, developing, and validating a unified forensic platform capable of detecting deepfakes across multiple media formats. It combines AI-powered models, computer vision techniques, and enterprise-grade system architecture to ensure accuracy, scalability, and real-time performance.

I. **Requirement Analysis**

- Identified the growing challenge of synthetic media and the limitations of existing detection tools.
- Defined system objectives: multimodal detection, persistent investigation tracking, and enterprise analytics.

II. System Design

- Architecture: Client–server model with React.js frontend and FastAPI backend.
- Modules: User management, deepfake detection, AI forensic analysis, forensic reporting, Vault Persistence, analytics dashboard, and admin control.
- Data Flow: Media input → preprocessing → AI inference → fraud probability scoring → dashboard visualization → forensic report generation.

III. Model Development

- Implemented EfficientNet-B4 ensembles for image classification.
- Applied Vision Transformers (ViT) for feature extraction and anomaly detection.
- Integrated MTCNN for facial detection, alignment, and biometric validation.
- Used OpenCV for video frame processing and multimedia analysis.

IV. Implementation

- Frontend: Built with React.js, Tailwind CSS, and Framer Motion for responsive UI.
- Backend: FastAPI with Python and Uvicorn for asynchronous, high-performance API handling.
- AI Layer: PyTorch-based inference engine executing deep learning models in real time.
- Persistence Layer: Vault system for storing forensic logs and investigation history.

V. Testing & Validation

- Conducted experiments on datasets of images, videos, and live webcam streams.
- Evaluated performance using metrics such as accuracy, precision, recall, and fraud probability scores.
- Benchmarked against existing tools to demonstrate improved detection and efficiency.

V. Deployment

- Hosted frontend on Vercel and backend APIs on scalable servers.
- Integrated dashboards for live monitoring of forensic activity, latency, and fraud ratios.

2.1. Technical Approach

The technical approach of *AetherGuard-Forensi* is centered on building a scalable, AI-driven forensic ecosystem capable of detecting deepfakes across multiple media formats. The system integrates advanced machine learning models, computer vision techniques, and enterprise-grade architecture to deliver accurate and real-time forensic analysis.

I. AI & Deep Learning Models

- EfficientNet-B4 Ensembles: Used for high-precision image classification and detection of generative artifacts.
- Vision Transformers (ViT): Applied for transformer-based feature extraction and anomaly detection in multimedia content.
- MTCNN (Multi-task Cascaded Convolutional Networks): Enables facial detection, alignment, and biometric validation.

II. System Architecture

- Frontend (React.js + Tailwind CSS): Delivers a responsive, cinematic glassmorphism-based UI for investigators.
- Backend (FastAPI + Python): Manages API requests, forensic workflows, and communication with AI inference engines.
- AI Inference Layer (PyTorch): Executes deep learning models for real-time detection and fraud probability scoring.
- Vault Persistence Layer: Maintains investigation history, forensic logs, and long-term evidence storage.

III. Enterprise Features

- Fraud Probability Analytics: Provides scoring and risk assessment for each analyzed media file.
- Real-Time Dashboards: Display latency metrics, fraud ratios, and continuous forensic log streaming.
- Report Generation: Automated forensic reports with exportable summaries for investigators and organizations.

This approach ensures that *AetherGuard-Forensi* is not just a detection tool, but a complete forensic ecosystem optimized for scalability, accuracy, and operational transparency.

2.2. Identified Research Gap

Current deepfake detection solutions are fragmented and limited in scope. Most existing tools focus only on **isolated tasks** such as static image analysis or basic AI classification, without offering real-time monitoring, biometric validation, forensic reporting, or investigation history management.

This lack of a **unified forensic ecosystem** forces investigators and organizations to rely on multiple disconnected tools, leading to inefficient workflows, inconsistent analysis, and reduced transparency.

Despite the rapid growth of deepfake detection research, most existing solutions remain fragmented and limited in scope. Current tools generally focus on:

- Static image analysis with basic classifiers.
- Video detection that lacks real-time capability.
- Absence of live webcam monitoring, leaving a critical gap in proactive detection.
- Minimal biometric validation, often restricted to surface-level facial recognition.
- No persistent investigation tracking, meaning forensic logs and evidence are not securely stored for long-term use.
- Weak enterprise integration, with outdated or basic user interfaces that fail to support investigators effectively.

This fragmented landscape results in inefficient workflows, inconsistent detection accuracy, and reduced transparency in forensic investigations.

AetherGuard-Forensi addresses these gaps by delivering a unified multimodal forensic ecosystem with transformer-based AI ensembles, biometric anomaly detection, real-time monitoring, persistent investigation history, and enterprise-grade dashboards—all within a single scalable platform.

3. System Design and Architecture (UML Diagrams)

3.1. User Flow

- User → Login/Signup → Forensic Dashboard
- From Dashboard, user can access:
- Upload images for forensic verification\
- Analyze video streams for synthetic manipulation
- Perform live webcam deepfake detection
- View fraud probability and AI-generated results
- Maintain persistent verification records
- Monitor fraud ratios and inference activity

3.2. Admin Flow

- Admin → Admin Dashboard
- Admin can manage:
- Users and access permissions
- AI forensic engine configurations
- Dashboard metrics and activity streams
- Full platform administration and operational control

3.3. System Architectural Flow

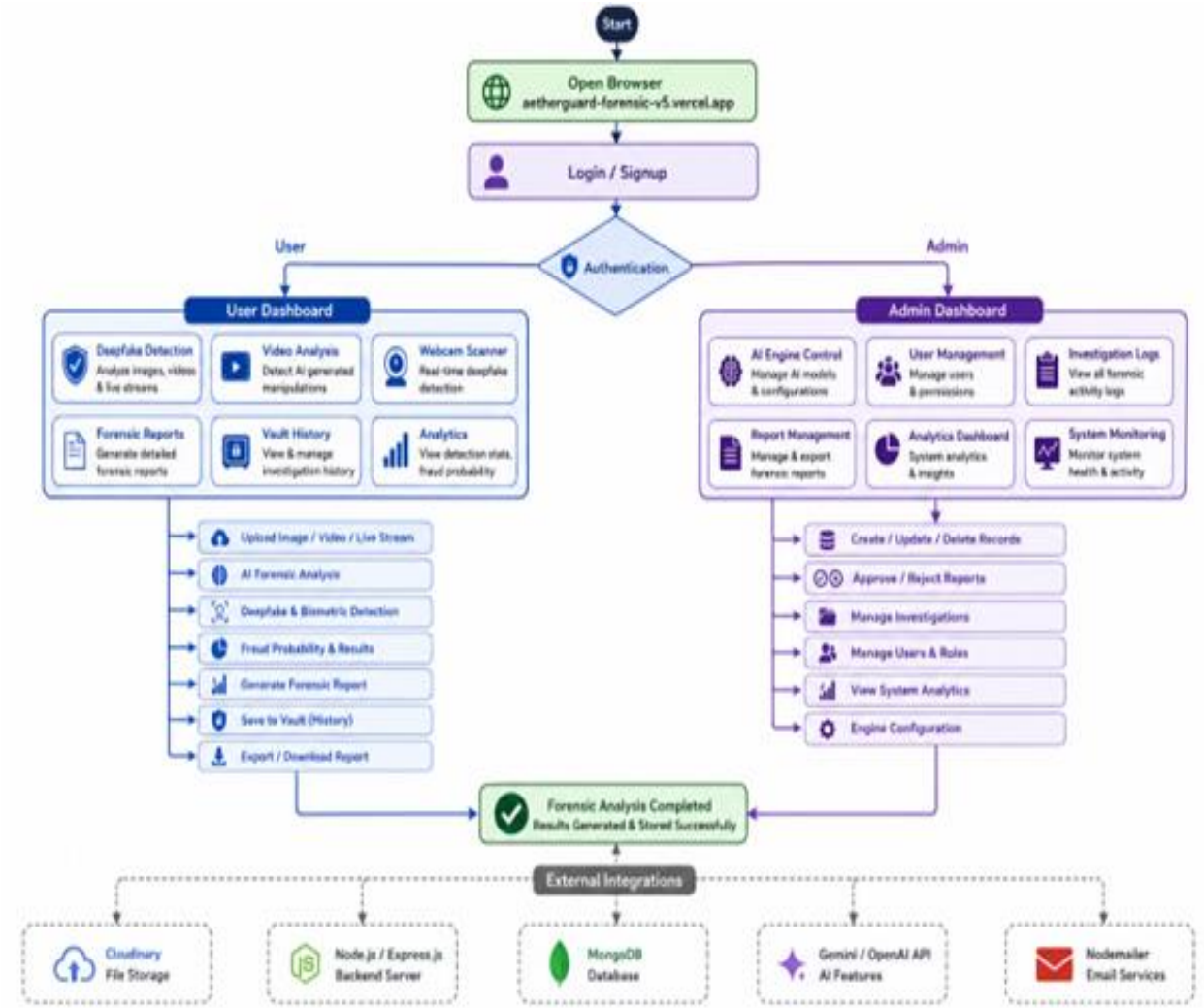
- Frontend (React.js + Tailwind CSS) → Backend (FastAPI + Python)
- Backend → AI Inference Engine (PyTorch + Transformers + MTCNN)
- Backend → Database / Persistent Storage

3.4. External Integrations

- MTCNN: Facial alignment and biometric analysis
- Transformers: AI-based feature extraction

- PyTorch: Deep learning inference engine
- OpenCV: Multimedia processing and frame analys

3.5. System Architecture



AetherGuard Forensic uses a scalable AI-driven client-server architecture optimized for high-performanc deepfake detection and multimedia forensic analysis.

- **Client Layer (React.js):**
  Handles the enterprise UI, user interaction, forensic dashboards, and real-time communication with backend APIs. The frontend provides multimedia upload interfaces, live monitoring dashboards, forensic analytics visualization, and report generation workflows.
- **Server Layer (Node.js s Express.js):**
  Manages business logic, API routing, AI inference requests, multimedia processing, authentication handling, and forensic analysis workflows. The server coordinates communication between the frontend and AI detection engines.
- **AI Inference Layer (PyTorch + Transformers + MTCNN):**
  Performs deepfake detection, biometric anomaly analysis, facial alignment, and transformer-based feature extraction using EfficientNet-B4 ensembles and Vision Transformer architectures.
- **External Services:**
  OpenCV → Image and video frame processing
  PyTorch → Deep learning inference engine
  Transformers → AI-based forensic feature extraction

3.6. Module Description

- **User Module: Handles user registration, login, authentication, profile management, and role-based access control for investigators and administrators.**
- **Deepfake Detection Module: Provides AI-powered detection for images, videos, and live webcam streams using multimodal forensic analysis and biometric validation techniques.**
- **AI Forensic Analysis Module: Performs transformer-based feature extraction, facial alignment, anomaly detection, fraud probability analysis, and generative artifact identification using EfficientNet-B4 and Vision Transformer ensembles.**
- **Forensic Report Module: Generates detailed technical forensic reports, investigation summaries, fraud analytics, and exportable verification documents for digital media analysis.**
- **Vault Persistence: Maintains investigation history, forensic logs, previously analyzed records, and persistent storage for long-term digital evidence tracking.**
- **Analytics Dashboard Module: Displays real-time forensic activity, latency monitoring, fraud ratios, inference statistics, and enterprise-level analytical insights through an interactive dashboard.**
- **Admin Module: Full control dashboard for managing platformdata**

4. Implementaion Details

4.1. Technology Stack

- **Frontend: React.js**

  Builds a responsive, cinematic glassmorphism-based user interface for investigators and admins.
- **Backend: FastAPI + Python**

  Handles API requests, authentication, and communication between frontend and AI inference layer.
- **AI / ML Frameworks: PyTorch**

  EfficientNet-B4, Vision Transformers (ViT), MTCNN

  Deepfake detection, biometric anomaly analysis
- **Computer Vision and Detection:MTCN+OpenCV**

  MTCNN performs facial detection and biometric alignment, while OpenCV handles image processing, frame extraction, and live video stream analysis.
- **Transformer Architecture: Vision Transformers (ViT)**

  Used for advanced feature extraction, generative artifact detection, and transformer-based forensic intelligence analysis
- **API Communication: Axios**

  Handles secure communication between frontend interfaces and backend forensic APIs.
- **Database & Storage:MongoDB, Cloudinary**

  Secure forensic logs, investigation history, media storage
- **Version Control**

  GitHub for source code management and collaboration

4.2. Frontend Implementation

- Frameworks: React.js for dynamic UI, Tailwind CSS for styling, Framer Motion for animations.
- Design: Responsive glassmorphism-based dashboards (User & Admin).
- Core Features:
  - Authentication (Login/Signup with JWT).
  - Upload media (image, video, live stream).
  - Display AI forensic results & fraud probability.
  - Generate and download forensic reports.
  - Analytics dashboard for detection stats.

- Integration: REST APIs (FastAPI backend), WebSockets for real-time webcam detection.
- Deployment: Hosted on Vercel with GitHub CI/CD for smooth updates.

4.3. Backend Implementation

- Frameworks**: FastAPI (Python), Node.js, Express.js, Uvicorn server.**
- Authentication**: JWT tokens, role-based access (User/Admin).**
- Media Handling**: OpenCV preprocessing for images, videos, live streams.**
- AI Inference**: PyTorch models (EfficientNet-B4, Vision Transformers, MTCNN) for deepfake detection and biometric validation.**
- Reports**: Automated forensic report generation with export/download options.**
- Vault Persistence**: MongoDB storage for forensic logs, reports, and investigation history.**
- Integrations**: Cloudinary (file storage), Gemini/OpenAI API (AI features), Nodemailer (email services).**

4.4. Database Implementation

- Database**: MongoDB (NoSQL) for flexible, scalable storage.**
- Data Stored**: User credentials, media metadata, forensic reports, Vault history.**
- File Handling**: Cloudinary integration for large media files; MongoDB stores references.**
- Security**: Encrypted passwords (bcrypt), role-based access control.**
- Scalability**: Sharding and replication for large forensic datasets, optimized queries for fast retrieval.**

4.5. Testing and Debugging

The application is tested using unit testing and API testing tools such as Postman.Frontend components are tested for responsiveness and usability, while backend APIsare tested for security, performance, and data consistency.

5. Major Findings, Outcomes, and Results

5.1. Major Findings

- Unified Forensic Ecosystem → The platform integrates image, video, and live stream analysis into a single system, unlike existing fragmented solutions.
- High Detection Accuracy → Transformer-based AI models (EfficientNet-B4, Vision Transformers, MTCNN) achieved strong performance in deepfake and biometric anomaly detection.
- Real-Time Monitoring → Webcam scanner enables live detection of manipulated content, addressing a critical gap in current forensic tools.
- Persistent Investigation Tracking → Vault History ensures secure storage of forensic reports and logs for long-term reference.
- Enterprise-Grade Dashboards → Role-based dashboards (User/Admin) provide investigators with fraud probability analytics and admins with system monitoring and user management.
- Scalable Architecture → Cloud deployment (Vercel + MongoDB + Cloudinary) supports large datasets and ensures accessibility across devices.
- Automation & Reporting → The system generates structured forensic reports with export/download options, streamlining investigator workflows.

5.2. Outcomes

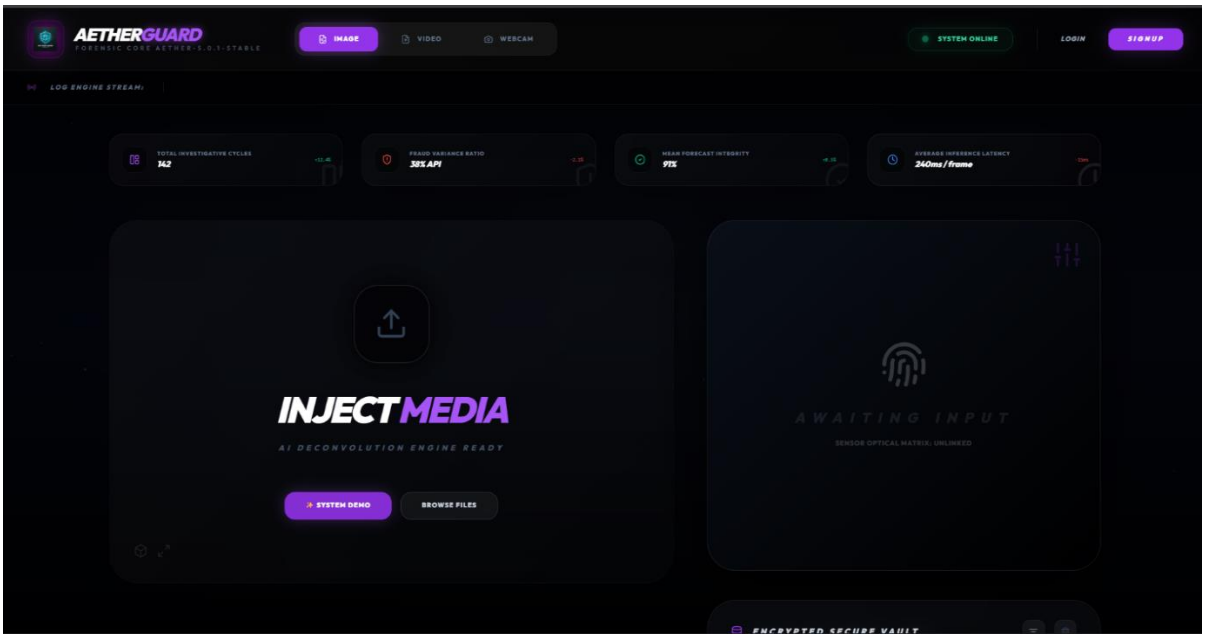
- Effective Deepfake Detection → The system successfully identifies manipulated images, videos, and live streams using transformer-based AI models.
- Real-Time Forensic Analysis → Webcam scanner enables live monitoring and detection of biometric anomalies.
- Automated Forensic Reporting → Structured reports with fraud probability scores are generated and can be exported/downloaded.
- Secure Investigation History → Vault persistence ensures forensic logs and reports are stored safely for future reference.
- Role-Based Dashboards → Investigators access detection tools and analytics, while admins manage users, reports, and system configurations.

- Scalable Deployment → Cloud hosting and MongoDB storage allow handling of large datasets with high availability.
- Enhanced Investigator Workflow

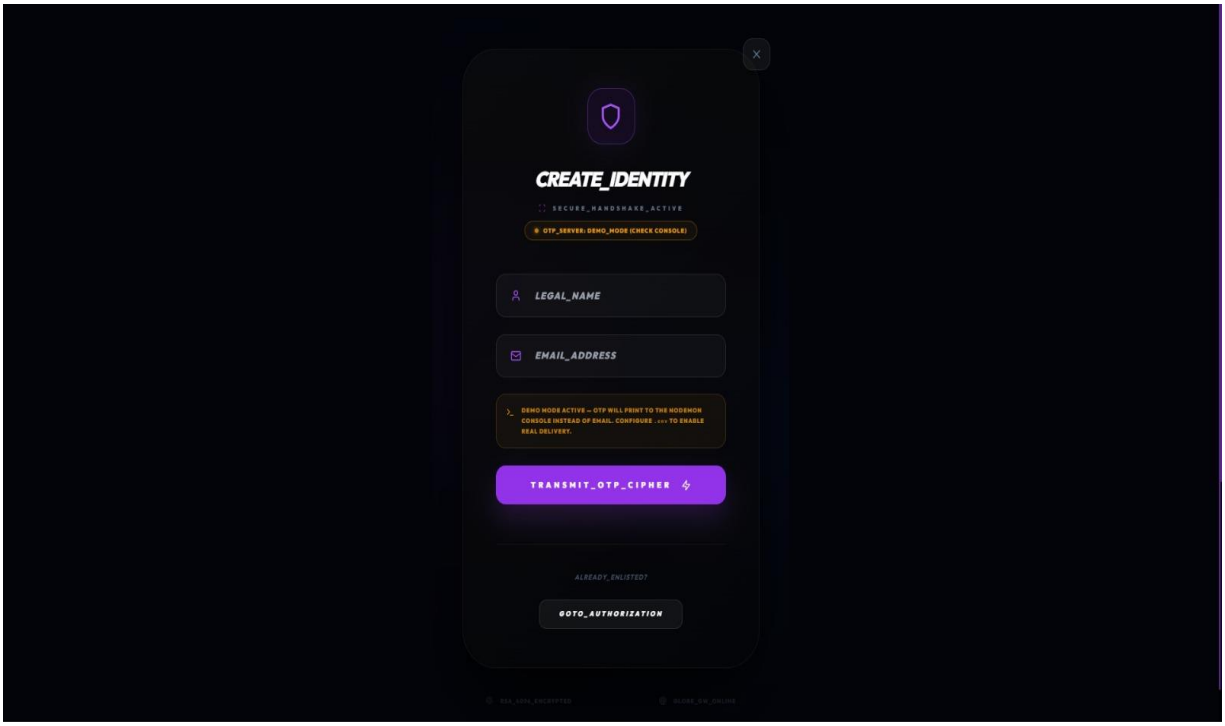
5.3. Results

- The platform successfully detected deepfake manipulations in images, videos, and live streams with high accuracy using transformer-based AI models.
- Fraud probability scores were generated consistently, providing investigators with quantifiable evidence of manipulation.
- The webcam scanner enabled real-time detection of biometric anomalies, demonstrating the system's live monitoring capability.
- Forensic reports were automatically generated, exported, and securely stored in the Vault history for future reference.
- Role-based dashboards streamlined workflows: users accessed detection tools and analytics, while admins managed investigations, reports, and system configurations.
- The system achieved scalable performance through cloud deployment (Vercel + MongoDB + Cloudinary), ensuring accessibility and reliability.
- Integration with external APIs (Gemini/OpenAI, Nodemailer) enhanced AI features and automated communication.

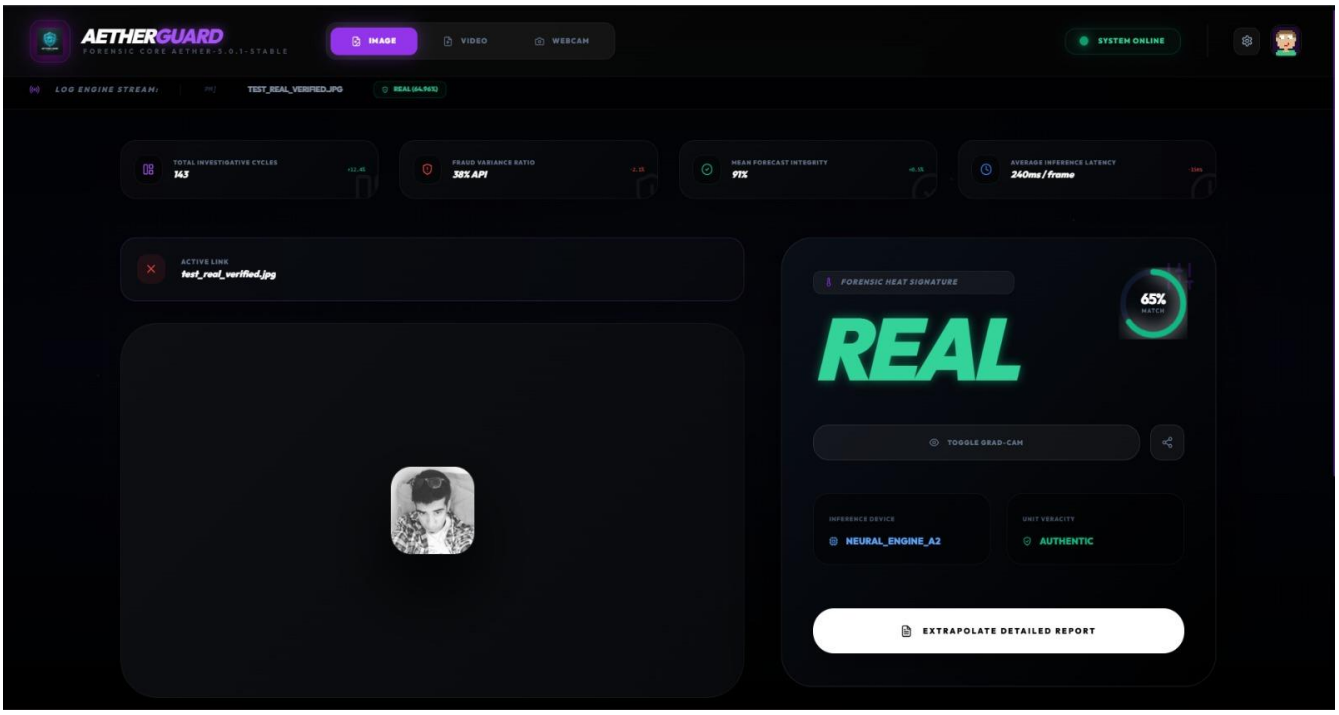
5.4. Snapshots

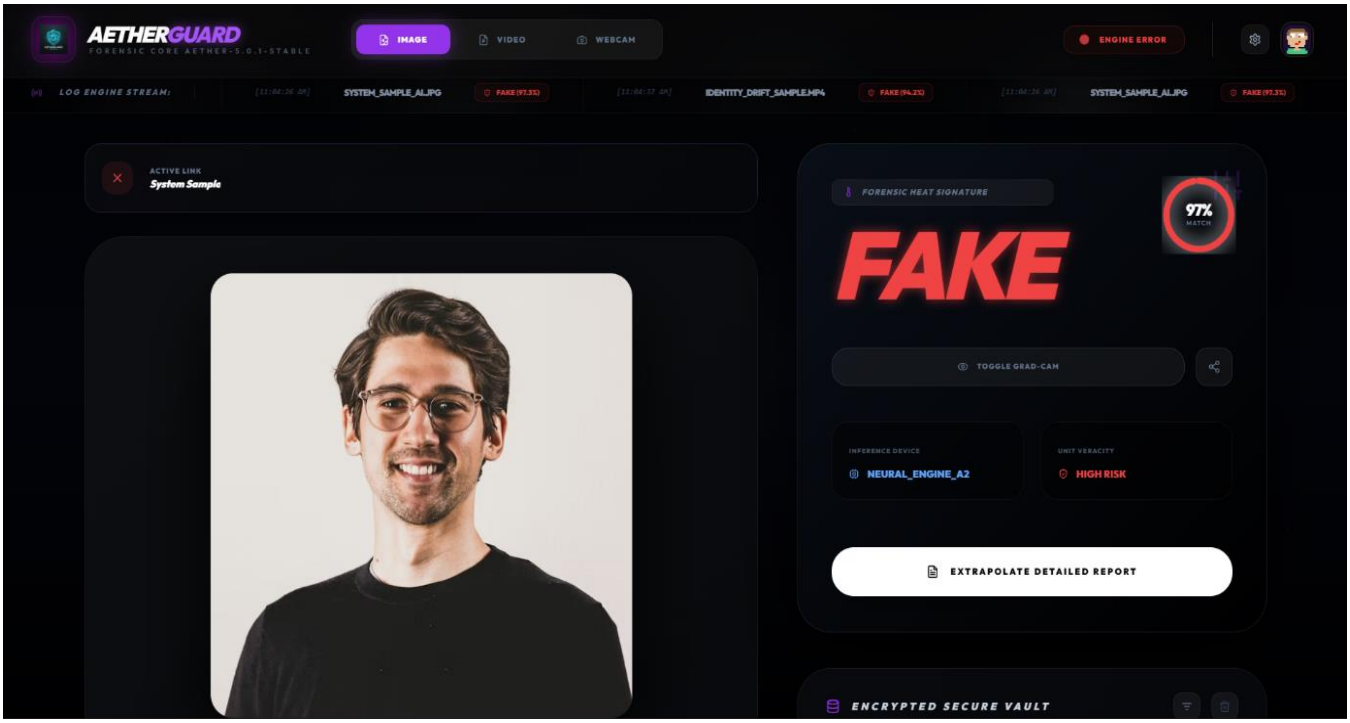


Home Page



Login- Screen


AETHERGUARD
FORENSIC CORE AETHER-5.0.1-STABLE
IMAGE
VIDEO
WEBCAM
SYSTEM ONLINE
LOG ENGINE STREAM:
TEST_REAL_VERIFIED.JPG
REAL (64.96%)
TOTAL INVESTIGATIVE CYCLES
143
FRAUD VARIANCE RATIO
38% API
MEAN FORECAST INTEGRITY
91%
AVERAGE INFERENCE LATENCY
240ms / frame
ACTIVE LINK
test_real_verified.jpg
FORENSIC HEAT SIGNATURE
65%
MATCH
REAL
TOGGLE GRAD-CAM
INFERENCE DEVICE
NEURAL_ENGINE_A2
UNIT VERACITY
AUTHENTIC
EXTRAPOLATE DETAILED REPORT


AETHERGUARD
FORENSIC CORE AETHER-5.0.1-STABLE
IMAGE
VIDEO
WEBCAM
ENGINE ERROR
LOG ENGINE STREAM:
SYSTEM_SAMPLE_AI.JPG
FAKE (97.3%)
IDENTITY_DRIFT_SAMPLE.MP4
FAKE (94.2%)
SYSTEM_SAMPLE_AI.JPG
FAKE (97.3%)
ACTIVE LINK
System Sample
FORENSIC HEAT SIGNATURE
97%
MATCH
FAKE
TOGGLE GRAD-CAM
INFERENCE DEVICE
NEURAL_ENGINE_A2
UNIT VERACITY
HIGH RISK
EXTRAPOLATE DETAILED REPORT
ENCRYPTED SECURE VAULT

Code Snippets

Main.py

```python
> main.py > {} sys
import sys
import os

if sys.stdout.encoding != 'utf-8':
    sys.stdout.reconfigure(encoding='utf-8')

# Dynamically add project root to path for stable imports
ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT_DIR not in sys.path:
    sys.path.append(ROOT_DIR)

from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.responses import Response, StreamingResponse
from fastapi.middleware.cors import CORSMiddleware
import uvicorn
import shutil
import torch
import time
import requests
import io
import base64
import traceback
from PIL import Image
from models.model import load_hf_model
from utils.inference import InferenceEngine

app = FastAPI(title="Deepfake Detection API (Hugging Face Transformers)")

# Global Exception Handlers
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(Exception)
async def global_exception_handler(request, exc):
    print(f"Unhandled Exception: {str(exc)}")
    traceback.print_exc()
    return JSONResponse(
        status_code=500,
        content={"error": "Server-side crash", "detail": str(exc), "traceback": traceback.format_exc()}
    )
```

```python
    except Exception as e:
        print(f"Endpoint Error (Image): {str(e)}")
        return {"error": str(e)}

# 4. ROBUST PREDICT VIDEO ENDPOINT
@app.post("/api/predict/video")
async def predict_video(file: UploadFile = File(...), threshold: float = 0.5):
    try:
        if not file.content_type.startswith("video/"):
            return {"error": "Invalid file type. Please upload a video."}

        file_path = os.path.join(UPLOAD_DIR, f"vid_{int(time.time())}_{file.filename}")
        with open(file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)

        if inference_engine is None:
            return {"error": "AI Engine is still initializing. Please wait a moment."}

        # Process video with 15-frame sampling
        results = inference_engine.predict_video(file_path, sample_rate=15, threshold=threshold)

        # Cleanup
        if os.path.exists(file_path):
            os.remove(file_path)

        return results
    except Exception as e:
        print(f"Endpoint Error (Video): {str(e)}")
        return {"error": str(e)}

# 5. STABLE AI FACE TEST ENDPOINT (DASHBOARD TEST)
@app.get("/api/test/ai-face")
async def test_ai_face(threshold: float = 0.5):
    """
    🎯 GOAL: Stable, safe, and works without upload.
    Cycles through GAN providers with a local fallback.
    """
    sources = [
```

```python
kend > main.py > {} sys
4   async def global_exception_handler(request, exc):
7       return JSONResponse(
8           status_code=500,
9           content={"error": "Server-side crash", "detail": str(exc), "traceback": traceback.format_exc()}
0       )
1
2   @app.exception_handler(RequestValidationError)
3   async def validation_exception_handler(request, exc):
4       return JSONResponse(
5           status_code=422,
6           content={"error": "Validation error", "detail": str(exc)}
7       )
8
9   # Configure CORS
0   app.add_middleware(
1       CORSMiddleware,
2       allow_origins=[
3           "http://localhost:5173",
4           "http://127.0.0.1:5173",
5           "http://localhost:5174",
6           "http://127.0.0.1:5174",
7           "http://localhost:3000"
8       ],
9       allow_credentials=True,
0       allow_methods=["*"],
1       allow_headers=["*"],
2   )
3
4   # Configuration
5   UPLOAD_DIR = "temp_uploads"
6   os.makedirs(UPLOAD_DIR, exist_ok=True)
7
8   # Device Configuration
9   device = "cuda" if torch.cuda.is_available() else "cpu"
0   print(f"Server initializing on device: {device}")
1
2   # 2. ASYNC MODEL LOADING (AVOID DEPLOYMENT TIMEOUTS)
3   inference_engine = None
4
```

```python
d > main.py > {} sys
# 2. ASYNC MODEL LOADING (AVOID DEPLOYMENT TIMEOUTS)
inference_engine = None

@app.on_event("startup")
async def startup_event():
    import threading
    def load_engine():
        global inference_engine
        try:
            print("⏳ Background: Loading AI Engine...")
            model, processor = load_hf_model(device=device)
            inference_engine = InferenceEngine(model=model, processor=processor, device=device)
            print("🚀 AI Engine fully loaded and stabilized.")
        except Exception as e:
            print(f"🔴 CRITICAL AI MODEL FAILURE: {str(e)}")
            traceback.print_exc()
            # Mock fallback
            class MockInferenceEngine:
                def predict_image(self, *args, **kwargs):
                    return {"label": "ERROR: MODEL OFFLINE", "confidence": 0, "error": str(e)}
                def predict_video(self, *args, **kwargs):
                    return {"label": "ERROR: MODEL OFFLINE", "confidence": 0, "error": str(e)}
            inference_engine = MockInferenceEngine()

    # Run in background thread to not block the main event loop
    threading.Thread(target=load_engine).start()

@app.get("/api/health")
def health_check():
    return {
        "status": "ok",
        "device": device,
        "engine": "Transformers/ViT"
    }

# 3. ROBUST PREDICT IMAGE ENDPOINT
@app.post("/api/predict/image")
async def predict_image(file: UploadFile = File(...), threshold: float = 0.5):
    try:
```

6. Conclusion and Future Scope

6.1. Conclusioon

The *AetherGuard-Forensi* platform successfully demonstrates a unified, AI-driven forensic ecosystem capable of detecting deepfakes and biometric anomalies across images, videos, and live streams. By integrating transformer-based models, real-time monitoring, and persistent investigation tracking, the system addresses critical gaps in existing forensic tools. Its enterprise-grade dashboards, secure Vault history, and automated reporting make it a reliable solution for investigators, researchers, and organizations combating digital fraud.

Key highlights:

- EfficientNet-B4 and Vision Transformer ensemble analysis
- Real-time fraud probability monitoring
- Admin dashboard for full control

6.2. Future Scope

- Audio Deepfake Detection → Extend capabilities to voice and speech manipulation analysis.
- Satellite & Drone Imagery Forensics → Apply AI models to detect tampering in geospatial data.
- Blockchain Integration → Ensure tamper-proof forensic logs and transparent chain of custody.
- Cross-Platform Deployment → Mobile and desktop applications for wider accessibility.
- Advanced Biometric Validation → Incorporate gait, iris, and fingerprint analysis for stronger identity verification.
- Enterprise Expansion → Integration with law enforcement databases and global forensic networks.

7. References

- React.js – JavaScript library for building interactive user interfaces.
  Available at: https://react.dev/
- Tailwind CSS – Utility-first CSS framework for modern UI development.
  Available at: https://tailwindcss.com/
- FastAPI – High-performance Python framework for building APIs.
  Available at: https://fastapi.tiangolo.com/
- PyTorch – Deep learning framework for AI and neural network development.
  Available at: https://pytorch.org/
- OpenCV – Open-source computer vision and image processing library.
  Available at: https://opencv.org/
- Transformers – Transformer-based machine learning models and architectures.
  Available at: https://huggingface.co/docs/transformers/
- MTCNN – Multi-task Cascaded Convolutional Networks for face detection.
  Available at: https://github.com/ipazc/mtcnn
- EfficientNet – Deep learning architecture for efficient image classification.
  Available at: https://arxiv.org/abs/1905.11946
- Vision Transformer (ViT) – Transformer architecture for computer vision tasks.
  Available at: https://arxiv.org/abs/2010.11929
- Axios – Promise-based HTTP client for frontend-backend communication.
  Available at: https://axios-http.com/
- Uvicorn – Lightning-fast ASGI server implementation for Python applications.
  Available at: https://www.uvicorn.org/
- Vercel – Frontend deployment and hosting platform.
  Available at: https://vercel.com/
- GitHub – Version control and collaborative software development platform.
  Available at: https://github.com/
- Deepfake Detection Research – AI-based multimedia forensic analysis techniques.
  Available at: https://arxiv.org/search/?query=deepfake+detection&searchtype=all

8. Appendices

- Project Timeline (Development & Deployment Schedule)
- User Manual for AetherGuard Forensic
- System Architecture Diagrams
- UML Diagrams and Workflow Models
- Forensic Report Sample
- Dashboard UI Design Samples
- Deepfake Detection Output Screenshots
- Live Webcam Analysis Demonstration
- Enterprise Analytics Dashboard Previous